AgriAgent Team Tasks and Test Cycle 1

Step-by-step instructions for Sahil, Sumant and Amit · version 2 · test cycle 1 due Tuesday 27 October

AgriAgent Final Year Project · 20 October 2026

Read this first (version 2, 20 October). Since 17 October the project gained a measured improvement (weed recall on unseen photos from 1.8% to 77.9%), an escalation agent, a farmer summary in English/Hindi/Marathi, and a corrected herbicide table. What happened and why: `docs/AgriAgent_Week4_Dossier.pdf`. **Testing is now the team's job**: this document gives every test case with exact steps and the exact expected result. You do not write code or Typst: you run commands, use the app, compare with the expected result, record what you saw, and file a GitHub issue for every difference. Version 1 tasks that are already done (the herbicide table check SU-2, the agent re-run SU-5) are removed; the others are kept below.

Deadlines. Part A, set-up and understanding: Wednesday 21 October, 23:59 (unchanged). **Part B, remaining Week 4 tasks: Sunday 25 October. Part C, test cycle 1: Tuesday 27 October, 23:59. Part D, regression cycle: Wednesday 4 November.** Code freeze is Thursday 5 November. Post progress in the group every evening you work on this (done / not done / blocked). Blocked for more than two hours: post the exact error message.

1 Where to put your work

What	Where (shared Drive folder `agriAgentShared`, called "AgriAgent Evidence" below)
Test results	Google Sheet **AgriAgent Test Cycle 1** (in the shared folder): one row per test case is already there. Fill `result` (pass / fail / blocked), `what you saw`, `evidence file`, `issue no.`, `tester`, `date`.
Screenshots, videos	Your own sub-folder (`Sahil/`, `Sumant/`, `Amit/`), file name = test ID, e.g. `ST-07.png`.
Other sheets	Google Sheet **AgriAgent Test Results**, one tab per task ID (SA-1, SA-2, ...).
Report content	Google Doc **AgriAgent Report Content**, one heading per task ID.
Test photos	`shared/test_photos/` in the Drive folder: six real field photos used by the test cases.
Defects	A GitHub issue per defect (template in Section 7).

2 Part A · everyone · set-up and understanding (Wednesday 21 October)

2.1 E-1 · Set up the project (about 1.5 h)

1. Terminal (Windows: Command Prompt). If you have the repository: `git checkout main` then `git pull`. If not: `git clone https://github.com/Chaitanya-666/agriAgent.git` and `cd agriAgent`.
2. `python -m venv .venv`, then activate (Windows `.venv\Scripts\activate`, Linux/macOS `source .venv/bin/activate`).
3. `python -m pip install -e ".[dev]"` (15–20 minutes the first time).
4. Copy the settings file (Windows `copy .env.example .env`, Linux/macOS `cp .env.example .env`); put **your own** free Groq key from `console.groq.com/keys` after `GROQ_API_KEY=`. Never share or commit `.env`.
5. `python scripts/check_providers.py` must show `[PASS] groq`.

6. `python -m unittest discover tests` must end with `OK`.
7. `streamlit run app/app.py` must open a browser tab with a yellow **MOCK OUTPUT** banner.

Submit: three screenshots (PASS line with the key hidden, the test `OK` line, the app) in your Drive folder as `E-1a.png`, `E-1b.png`, `E-1c.png`. **Done when:** all three are there.

2.2 E-2 · Understand the project (about 3 h)

Read `docs/AgriAgent_Week4_Dossier.pdf`, `docs/AgriAgent_Week3_Dossier.pdf`, `docs/AgriAgent_Project_Map.pdf` and Guide 2 §10. Then answer in your own words (3–5 lines each) under heading E-2 in the Report Content doc:

1. Name the pipeline stages. Which are AI models, which are plain rules?
2. Why does a Grounding DINO label "weed" not mean the model understood the plant is a weed?
3. A weed mask has confidence 0.60. Is it sprayed by the gate alone? Which file decides?
4. Why was `eval_manifest_v1` thrown away, and what is `eval_manifest_v2_holdout` for?
5. What do "crop pixels sprayed" and "weed recall" measure? Why only sugar beet for the first?
6. How do the plant rules decide crop or weed for a green patch? Name the four cases.
7. Why do the plant rules work on sugar beet but not on the cotton close-ups?
8. What does the escalation agent do, and why did it never approve a weed?
9. Why did one wrong rice row in the herbicide table matter (dose and waiting period)?
10. How can you tell on screen whether a result is REAL or MOCK?

Done when: all ten answers are in the doc.

2.3 SU-1 · Report format (Sumant, overdue)

Get the department's report format (template or written rules) and upload it to `Sumant/SU-1`.

3 Part B · remaining Week 4 tasks (Sunday 25 October)

ID	Owner	Task (full steps as in version 1, unchanged)
SA-1	Sahil	Check the 38 evaluation images' labels by eye (`python scripts/build_eval_manifest_v2.py`, then `python scripts/show_manifest_labels.py`; needs a Kaggle token in `.kaggle/access_token`). Tab SA-1: image id, every crop has a green box, every weed a red box, any box on bare soil, notes.
SA-2	Sahil	Hand-compute the three 5 × 5 examples (crop pixels sprayed, weed recall, savings) and post a photo of the working; then compare with test case VT-02.
SA-4	Sahil	Judge the 20 blind patches in `shared/t2_2_patches/t2_2_blind/` as crop / weed / can't tell, post "SA-4 answers submitted", then fill the answers from the key Chaitanya sends.
SA-5	Sahil	Pick 15 hard sugar beet photos not in the evaluation set (weed touching crop, very small weeds, shadow, crop at the edge, many weeds); tab SA-5. They are used in VT-13.
SA-6	Sahil	Report content: data and evaluation (datasets, why v1 was rejected, how v2 and the held-out set were built, the metrics, the size bias), with sources.
SU-6	Sumant	Report content: problem statement (5 bullets with sources) and the six papers listed in version 1 (what they did, dataset, key number, how we differ).
AM-1	Amit	Re-test closed issues #6–#13 with your own steps on `main`; comment PASS or FAIL with a screenshot on each issue, reopen on FAIL (now test case RT-01).
AM-3	Amit	Screen recording of one real-mode run in the app (Win + G), start to GeoJSON download.

AM-5	Amit	Google Slides skeleton: Problem · Research question · Pipeline · Safety gate · Plant rules · Data · Results · Escalation · Testing · Limits · Team.

4 Part C · test cycle 1 (Tuesday 27 October)

Run every case assigned to you on the latest `main` (`git pull` first) and fill its row in the **AgriAgent Test Cycle 1** sheet. **Mock** = the "Mock Mode" box in the sidebar ticked; **real** = unticked (the first real run downloads about 1.6 GB of model weights; on a laptop without an NVIDIA GPU each photo takes about 10–60 seconds). Test photos are in Drive `shared/test_photos/`: `beet_X-10-0.png` (4 beets, 2 tiny weeds: the rules work well here), `beet_X2-30-0.png` (3 beets, many tiny weeds: a **known failure**, two beets are partly sprayed), `beet_X3-430-0.png` (two clear crop rows), `beet_X3-130-0.png` (beets only, no weeds), `cotton_CM_17.jpg` and `cotton_CM_11.jpg` (cotton-field close-ups of weeds).

4.1 Sumant · system tests (does it work for a user?)

ID	Steps	Expected result (pass if all of it is true)
ST-01	Start the app, mock, sample photo.	Yellow banner starting "MOCK OUTPUT: these boxes, masks and the diagnosis are synthetic test data" above the four number cards.
ST-02	Same screen, scroll to "Summary for the farmer".	Blue box starting "This is test data, not a real field result." and a "Language" selector.
ST-03	Mock. Clear the "Zero-Shot Detection Prompt" box completely, press Enter.	Red error "The detection prompt is empty. Enter what to look for, e.g. 'weed . cotton plant .'"; no number cards.
ST-04	Mock. Change "Crop Context" to Sugar Beet, then Wheat, then Rice.	Prompt box changes to `weed . sugar beet plant .`, then `weed . wheat plant .`, then `weed . rice plant .`.
ST-05	Mock. Prompt `cotton . weed .`, download the GeoJSON, open it in a text editor.	`"features": []` and `"status"` starting "No weed zones approved".
ST-06	Mock. Confidence slider to 0.90, download GeoJSON.	"Target Weed Clusters" card shows 1; GeoJSON has exactly one feature with `"confidence": 0.91`.
ST-07	Mock, default settings, download GeoJSON.	No feature has `active_ingredient`; metadata `"statutory_status": "no_prescription"`; `"coordinate_note"` starts "Not GPS".
ST-08	Mock. Tick "Force System 2", crop Cotton.	Status box says "MOCK DIAGNOSIS"; the word "VERIFIED" appears nowhere on the page.
ST-09	Mock. Force System 2 with Wheat, then Rice, then Sugar Beet.	Each shows "Status: NO STATUTORY EVIDENCE FOUND — ZERO CHEMICALS PRESCRIBED"; "Pyrithiobac" appears nowhere.

ST-10	Rename any `.txt` file to `test.jpg` and upload it.	Red “Image Decoding Error: …”; the app does not crash.
ST-11	Upload a phone photo larger than 5 MB (mock).	Result appears without a crash; note the seconds.
ST-12	Untick Mock Mode, keep the sample photo (first real run).	Green banner “REAL OUTPUT: Grounding DINO and SAM 2 ran on this image.”; no red error; note the total minutes including the download.
ST-13	Real. Crop Sugar Beet, upload `beet_X-10-0.png`.	REAL banner; the farmer summary says weed patches are marked; in the right-hand picture colour appears only on tiny plants, **not** on the four large beets. Screenshot.
ST-14	Real. Crop Sugar Beet, upload `beet_X3-130-0.png` (no weeds).	“Weed Infestation Area” below 2% and no colour on the beets. Then upload `beet_X2-30-0.png` and record which beets are coloured (known failure; record, do not file as a new defect).
ST-15	Real. Crop Cotton, upload `cotton_CM_17.jpg`.	No crash; record what is marked and what the summary says. (Known limit: few weeds are found on cotton.)
ST-16	Real, after ST-13: “Language” selector to Hindi, then Marathi.	Summary text changes to that language; caption “Reworded by the Groq model from the result above.” (Without a Groq key: English and “Built directly from the result above.”)
ST-17	Real, after ST-13: download GeoJSON and open it at `geojson.io`.	At least one feature with `"target": "weed patches (plant rules)"` shapes appear; `coordinate_note` says not GPS.
ST-18	Real. Look for the expander “Escalation agent: … withheld plant(s) reviewed” after ST-13 or ST-15 and open it.	If present: each plant shows a verdict (weed / crop / unsure), P(weed) and its evidence lines. Record whether it appeared.
ST-19	Real. Turn Wi-Fi off (models already downloaded), run `beet_X-10-0.png` again.	Vision still runs (REAL banner); the summary falls back to English text; no crash. A crash or endless spinner is a defect.
ST-20	Real. In `.env` set `GROQ_API_KEY=` (empty), restart the app, run ST-13 with “Force System 2”.	No crash; the page shows a result; any message about the missing diagnosis is acceptable. Restore your key afterwards.
ST-21	Terminal: `python scripts/run_live_pipeline.py --image <path to beet_X-10-0.png> --crop "sugar beet" --output st21.png`	Lines “MODE USED : detector=REAL, segmenter=REAL”, “Plant rules (green patches): {…}” and “Summary for the farmer”; `st21.png` has four panels.

ST-22	Same command with `--no-plant-rules`.	Status "No weed zones approved: nothing to spray …" (the old behaviour). Compare with ST-21 in one sentence.

4.2 Sahil · validation tests (is the output correct?)

ID	Steps	Expected result (pass if all of it is true)
VT-01	**5 task SA-1 (label check by eye).**	All 38 rows filled; every problem described.
VT-02	After SA-2: `python -m unittest tests.test_crop_protection -v`	11 tests "ok". Your hand results equal the test values: A 33.33 / 75.0 / 80.0, B 0.0 / 0.0 / 100.0, C 100.0 / 100.0 / 0.0. Any difference: say which is wrong and why.
VT-03	`python scripts/run_baseline.py --limit 10 --name repro_sahil` (needs SA-1's download).	For each of the 10 images, `plant_rules` and `gate_0.35` numbers in `data/outputs/baseline/repro_sahil/results.json` are within 1 percentage point of the same image in `docs/results/plant_rules_v3_20261018.json`.
VT-04	`python scripts/run_baseline.py --manifest data/manifests/eval_manifest_v2_holdout.json --name repro_holdout_sahil` (all 90 held-out photos; about 30–60 minutes on a laptop without an NVIDIA GPU).	The summary table in `data/outputs/baseline/repro_holdout_sahil/summary.md` matches `docs/results/holdout_plant_rules_v3_20261018.md` within 1 point for `plant_rules` and `gate_0.35` (77.9% weed recall, 8.8% crop pixels sprayed).
VT-05	Open the 10 figures of VT-03 (`figures/`). Panel 4 is the product setting (`pipeline_plant_rules`). For each, count large beets (green boxes in panel 1) that have any red in panel 4.	Your count matches `crop_plants_damaged` of `pipeline_plant_rules` in `results.json`; record every mismatch with the image name.
VT-06	In VT-04's `results.json`, find the images that have no weed (`weed_pixels` = 0). For each, note `crop_sprayed_pct` and `crop_plants_damaged` of `pipeline_plant_rules`, and look at its figure.	Each value matches the same image in `docs/results/holdout_plant_rules_v3_20261018.json` within 1 point. Also write one sentence per image on what the figure shows (beets coloured or not). Known failures elsewhere in the held-out set: `X2-290-0` (6 of 8 beets touched) and `X5-510-0` (4 of 4).
VT-07	Run VT-03 a second time with `--name repro_sahil_2`.	Every metric identical to the first run.
VT-08	In VT-03's `results.json` check every `metrics` entry.	`savings_pct` between 0 and 100; `sprayed_pct + savings_pct` = 100 (±0.01); percentages never above 100.

VT-09	Pick 4 rows of `docs/results/icar_table_check_20261020.md` (include rice pyrazosulfuron) and open the PDF at the printed page (PDF page = printed page + 8).	Dose, water, waiting period and weeds in the PDF match the "fix" column. Any mismatch is a high-severity defect.
VT-10	For ST-13's or ST-21's run, check every number and claim in the farmer summary against the printed results.	Nothing in the summary is absent from the results; no chemical or dose appears unless a prescription was printed.
VT-11	Open `docs/results/agent_v2_20261018.json`; take 10 entries of `agent_reviews`.	Verdict "weed" only when `p_weed` ≥ 0.85, "crop" only when ≤ 0.15, otherwise "unsure".
VT-12	**6 task SA-4 (blind patches).**	20 answers and your score posted.
VT-13	Run `run_live_pipeline.py --crop "sugar beet"` on each of SA-5's 15 hard photos; look at panel 4.	Per photo: crop plants coloured (count), visible weeds missed (count), one sentence. This is evidence, not pass/fail.

6.1 Amit · regression and demo (mostly testing)

ID	Steps	Expected result (pass if all of it is true)
RT-01	**7 task AM-1 (re-test #6–#13 with your own steps).**	A PASS or FAIL comment with a screenshot on each issue.
RT-02	Run TC-01…TC-16 of `docs/testing/System_Test_Runbook_Cycle0.pdf` again on `main`.	Fixed cases now pass: TC-03 (no herbicide in GeoJSON), TC-04 ("No weed zones approved"), TC-07 (empty-prompt error), TC-08 (one feature at 0.90), TC-10 (MOCK DIAGNOSIS), TC-11 ("no statutory evidence"), TC-13 (MOCK banner), TC-15 (real mode runs). Others unchanged.
RT-03	On Windows: `python -m unittest discover tests`, `python tests/e2e/runner.py --all`, `python scripts/verify_cycle0_issues.py`.	`OK`; "Passed: 35"; seven `[PASS]` lines.
RT-04	**8 task AM-3 (real-mode screen recording).**	One complete recording in `Amit/`.
RT-05	Go through `docs/guides/04_Demo_Playbook.pdf` step by step on your laptop.	Every step marked ok / problem; steps that no longer match the app listed.

9 Part D · regression cycle (Wednesday 4 November)

After the defects from cycle 1 are fixed: every tester re-runs every case that failed, plus five cases that passed (your choice), on the latest `main`, and adds a `regression result` column to the same sheet. The tester who filed a defect closes its issue after the re-test passes.

10 Templates and rules

10.1 GitHub issue for a defect

Title `[ST-07] what goes wrong, in one line` (use the test ID). Body: **Steps** (numbered), **Expected** (copy from this document), **Actual**, **Severity** (high: crop could be sprayed, mock shown as real, wrong chemical or dose, or the demo crashes; medium: wrong number or misleading screen; low: cosmetic), **Environment** (Windows/Linux, Python version, `git log -1` first 7 characters, mock or real), **Evidence** (screenshot attached).

10.2 Rules

- No “pass”, “done” or “verified” without a screenshot or pasted output.
- Record what you saw even when it passes; “pass” alone is not evidence.
- The tester who filed a defect closes it; nobody closes their own fix.
- Do not edit the manifests, change models, or add packages.
- If an AI tool helps you, understand what it did before you submit; the viva will ask.